Chair of Integrated Systems
School of Computation, Information and Technology
Technical University of Munich

Design of a Flow-Aware Packet Buffer for Hardware-Based Forward Error Correction

Master thesis

Author: Andreu Roca i Montserrat

Advisor: William Wulff
Supervisor: Prof. Dr. sc. techn. Thomas Wild

Submission date: August 24, 2026

Abstract

An abstract is defined as an abbreviated accurate representation of the contents of a document. – American National Standards Institute (ANSI)

Contents

List of Acronyms

AXI Advanced eXtensible Interface. 7, 8, 23–29, 33, 35, 36, 39, 40

CMAC Gigabit Media Access Control. 21

FEC Forward Error Correction. 4, 7, 9, 10, 16, 21–23, 38, 40

HBM High Bandwidth Memory. 8, 18, 21, 23–30, 33, 35

NIC Network Interface Card. 9, 21

List of Figures

List of Tables

1 Introduction

Reading this section should give the reader a understanding why he or she should read your thesis and what he or she can expect from it. The section should include the following parts.

1.1 Context and motivation

Any computer connected to a network contains some sort of network interface, usually in the form of a Network Interface Card (NIC) which acts as an interface between the physical layer (e.g. Ethernet or Wi-Fi), and the rest of the system.

As network speeds increased and more advanced features were employed in network processing, general purpose CPUs could not always keep up with the required rate and volume of packet processing. In modern high-speed applications network processing is therefore offloaded to the NIC, and the SmartNiC term was adopted to reflect the additional functionality.

A SmartNiC contains specific hardware that can perform tasks such as packet classification, filtering, flow handling, and FEC processing directly on the network interface. This specialized hardware provides a much higher efficiency and speed, which allows packets to be processed at or near line rate while freeing the CPU to focus on application-level computation.

1.1.1 Forward Error Correction over Groups of Packets

Forward Error Correction (FEC, and also called channel coding) is a type of digital signal processing that improves data reliability by introducing a known structure into a data sequence prior to transmission or storage. This structure enables a receiving system to detect and possibly correct errors caused by corruption from the channel and the receiver. As the name implies, this coding technique enables the decoder to correct errors without requesting retransmission of the original information. However, this approach requires additional processing and bandwidth on the communication channel.

FEC is often applied across multiple packets because redundancy is most useful when it is shared across a group, or block, of source packets. By encoding packets

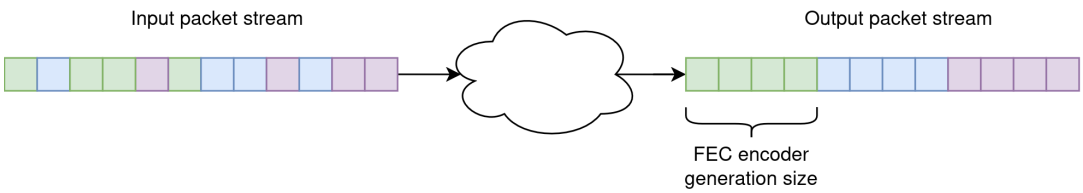

Figure 1.1: Packet reordering needed for FEC encoding

together, the sender can generate repair packets that allow the receiver to reconstruct one or more packets lost during transmission, rather than adding independent redundancy to every packet. This makes the available redundancy more efficient and can reduce the need for retransmissions, which is particularly valuable in unreliable, multicast, or high-loss networks. The trade-off is that larger FEC blocks generally require more buffering and can introduce additional decoding latency, since the receiver may need to collect enough packets from the block before recovery is possible. Standards such as the IETF's FEC framework and Raptor FEC scheme explicitly organize data into source blocks for this purpose [10].

Needless to say, the packets across which the encoding is performed must have the same destination, otherwise the additional information would be pointless. Because of this, network processors must detect packet flows and perform the encoding accordingly. It is important to remark that these FEC computations (or generations) are performed over groups of a fixed amount of packets, and that the size of the group is known as the generation size.

1.1.2 HyperNiC project and resilient worlds

<ASK WILLIAM ON WHAT TO PUT WHERE>

1.2 Problem definition

The problem to solve is simple at a surface level as illustrated in the diagram 1.1, where every color represents a different packet flow. Although packets arrive in order, they do not necessarily arrive in perfectly sized bursts belonging to a single flow. As stated previously, FEC calculations must be performed on a per-flow basis and over generation-sized blocks; therefore a mechanism is needed to collect and separate packets by flow, accumulate complete generations, and feed them to the FEC encoder in the required order and block size.

1.3 State of the art

There is relevant literature covering packet buffer architectures for high speeds (further details will be covered in the background chapter 2). The industry standard approach consists of a shared memory architecture, where a common pool of memory is used to store packets from multiple flows or queues, rather than allocating a dedicated buffer to each flow. Incoming packets are classified by flow and stored in available locations within the shared buffer, while per-flow queues maintain pointers or descriptors to the packets.

Some research showcases a generic packet buffer architecture [8], and others adapt said architecture to a specific context [6] [7], particularly in high speed network switches and routers. These are very useful starting points, but they do not cover many relevant implementation details when it comes to tracking flow status and the specifics of handling memory interfaces and latency, which will be discussed in the following chapters 3 and 4.

1.4 Methodology

The development of this module followed an iterative design methodology, in which the architecture was progressively refined through repeated cycles of design, implementation, and evaluation. Initially, different architectural approaches were investigated to determine how packets could be efficiently organized and buffered before being processed by the FEC encoder. Multiple approaches have been implemented and evaluated through functional simulation to verify its correctness and through synthesis to assess its hardware resource utilization, timing, and overall feasibility for the target platform.

Particular attention was given to the HBM management architecture, for which several approaches were explored to improve memory utilization, access efficiency, and scalability. The results obtained from each iteration informed subsequent design decisions, leading to modifications of the buffering, queueing, and memory-management mechanisms. The architecture presented in this thesis therefore represents the final iteration of this iterative development process, selected based on the functional and hardware-level evaluations performed throughout the design.

1.5 Scope and limitations

The packet buffer presented in this thesis should be considered a first synthesizable implementation of the proposed architecture. The design has been functionally tested and evaluated against the requirements defined for the scope of this work,

demonstrating that the main buffering and packet organization mechanisms operate as intended. However, further development would be required before the module could be integrated into a complete FPGA-based system. In particular, the current implementation could be extended with more scalable packet queue implementation, dynamically configurable FEC generation sizes, and mechanisms for handling different flow priorities. These aspects are outside the scope of the current implementation, but the implemented design intentionally facilitates these future modifications.

Furthermore, the scope of this thesis is limited to the transmit (TX) side of the packet buffer architecture. The corresponding receive (RX) side is therefore not implemented or evaluated as part of this work. Nevertheless, its design could follow a similar architectural approach to the TX side, allowing a significant portion of the existing design, including the buffering and memory-management mechanisms, to be reused. Consequently, the implementation presented here provides a foundation for extending the packet buffer architecture towards a complete TX/RX system in future work.

1.6 Report structure

The thesis is structured as follows:

1. **Introduction**: Introduces the context and motivation of the work, defines the research objectives, and provides an overview of the main concepts, challenges, and methodology.
2. **Background**: Presents the technical foundations relevant to the work, particularly regarding the shared memory architecture.
3. **Design**: Defines the system requirements and presents the architecture of the packet buffer.
4. **Implementation**: Describes in further depth the implementation of the hardware modules and control logic.
5. **Evaluation**: Details the verification and testing procedures performed in simulation and presents the resulting performance measurements and resource utilization.
6. **Discussion**: Discusses and interprets the obtained results, addressing the main issues encountered during development as well as identified bottlenecks, possible improvements, and future directions.
7. **Conclusion**: Summarizes the main findings of the work, discusses its limitations, and presents concluding remarks and possible directions for future work.

2 Background

The objective of a packet buffer is to temporarily store incoming packets and organize them by flows so that different consumers can ingest them. Let us consider the case of a network switch as an example where there are many output interfaces operating at the same time and each of them has provides access to a different subset of the network. If packet from flow A and B have to be routed through a different physical interface each to reach their destinations, then an agent needs to separate the flows so that they can be routed accordingly. This is in essence the job of a packet buffer.

Before presenting the proposed design, this section reviews existing approaches described in the literature. Particularly, it covers the shared-memory architecture, which is the industry standard approach to buffering and rerouting packets in a network processor. That being said, there are specific challenges that are unique to the problem described in the introduction chapter 1 which will be discussed here as well.

2.1 The shared-memory architecture

As stated previously, there is renowned research on packet buffer architecture [8] and their application in a specific context, [6] [7]. For the purposes of this work, the paper published by Queens University Belfast [8] is particularly relevant as it defines and implements a generic shared packet buffer design.

The proposed design follows a shared-memory architecture in which instead of buffering the packets in a per-flow basis where every flow has its own dedicated memory, a common pool of memory is shared across all of them. Then, logical queues store pointers to the packets stored in the memory, which keeps track of which packets belong to which flow; and a different queue stores pointers to unallocated memory locations. This way, when a packet arrives an unallocated memory location is popped, the data from said packet is written in that region, and the pointer to the region is pushed to the respective flow queue. When a packet needs to be consumed, the inverse process takes place, where a packet address is popped from the respective flow queue, the packet is read, and the memory space (now free) is returned to the unallocated location queue. Figure 2.1 shows an overview of this

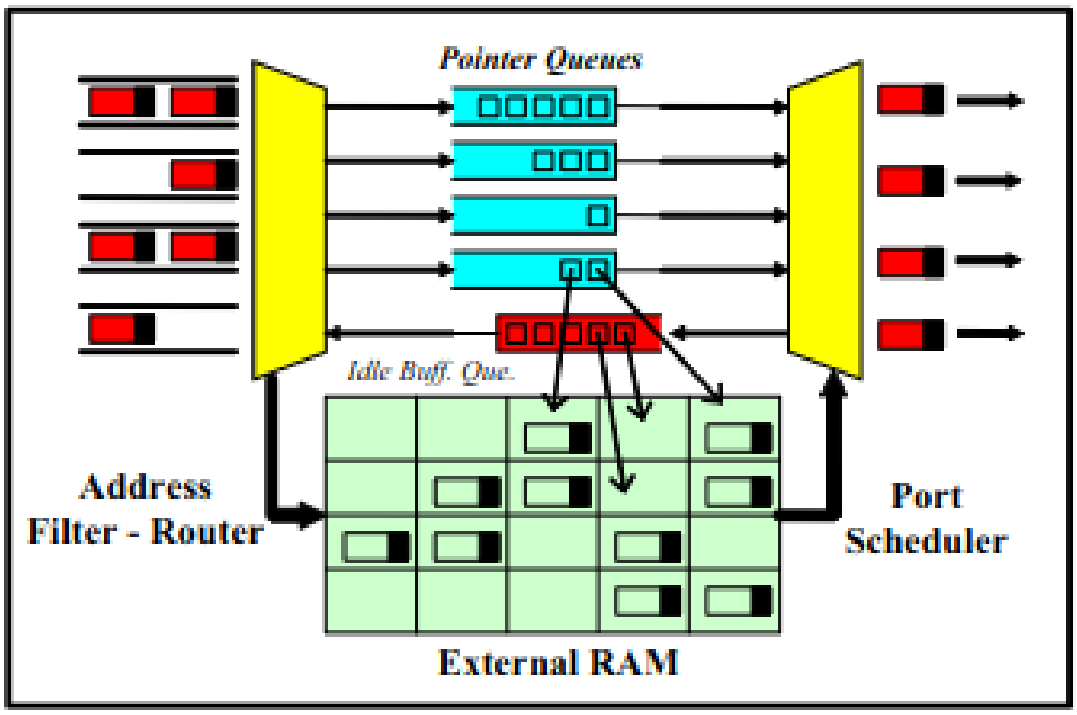

Figure 2.1: Shared-memory architecture (source [8])

architectural approach.

All in all, the architecture is essentially a dynamically allocated pool of fixed-size memory blocks combined with logical packet queues and buffer-management pointers, allowing variable-length packets from multiple queues to share the same physical memory.

Using descriptor queues instead of buffering packet data directly on a per-flow basis provides better memory utilization and greater flexibility since each flow only maintains a small queue of descriptors containing pointers and metadata for its packets. This allows memory to be dynamically allocated according to the actual traffic demand: a bursty flow can temporarily use more buffer space while idle flows do not waste their allocated capacity. It also naturally supports variable-sized packets, since packets can occupy different numbers of blocks in the shared memory.

The descriptor-based approach also separates packet storage from queue management. Pushing or popping a packet mainly involves manipulating small descriptors and pointers rather than moving the packet data itself, which reduces unnecessary data movement and makes flow management easier. Naturally, total memory size

also scales better with this approach and facilitates managing a larger number of concurrent flows.

2.2 Specific challenges

Here we say that the challenges architectures don't apply exactly to our problem:

- The papers don't mention handling of active/inactive flows.
- We have only 1 producer and consumer, potentially more consumers in the future.
- No mention of fixed-size stream of output packets needed
- No specifics on memory access latency to maintain throughput

3 Design

The main part of your thesis. Here you describe your approach to solve the problem you stated in the Introduction. Be as general as possible.

3.1 Context

As stated in the instroducton chapter 1 the potato

3.1.1 System requirements

These are the hard requirements:

- Support over X active flows at any time
- Generation size from 8 to 64
- Throughput of 100 Gb/s to feed Ethernet MAC
- Latency of under 10us to let FEC encoder dominate
- Configurable number of flows (compile time)
- Configurable generation size (compile time)

These are "nice to haves":

- Configurable priority of flows (dynamic)
- Configurable generation size (dynamic)

3.1.2 Throughput and latency

Here we explain that throughput (100Gb/s) and latency (¿1us) are the main

3.1.3 Assumptions and constraints

Here I should explain the design is only being simulated, that the target is to eventually synthesize it, and that there is still work to do to make it usable, mainly adding the configuration stuff. Remark that this is mostly the baseline platform to build from. Note that it is very expandable.

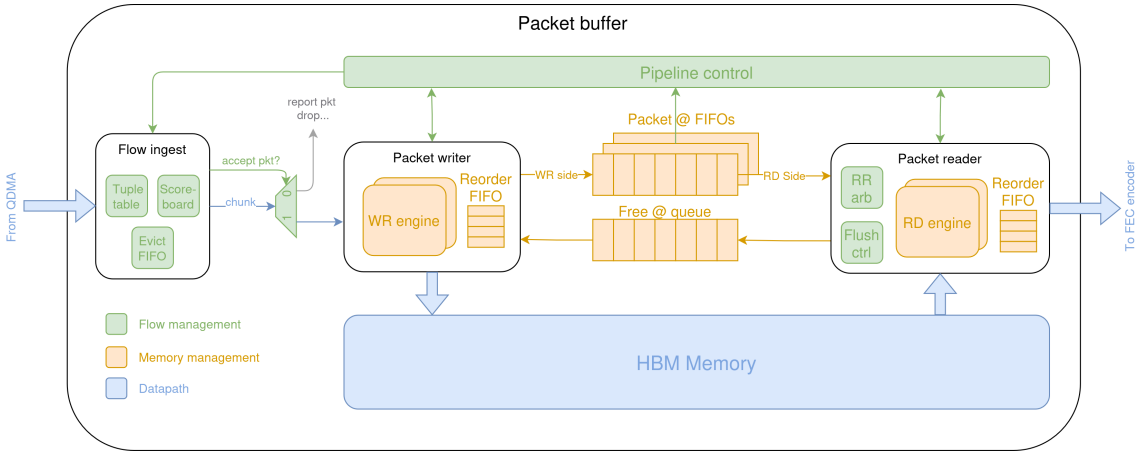

Figure 3.1: Overview of the Packet Buffer

3.2 System design

As covered in the background chapter 2, the design is built with a shared-memory architecture. Figure 3.1 depicts a high-level architectural view of the system. The subsequent sections break down the different stages roughly following the order of the data path.

3.2.1 Flow ingest

This is the first stage in the data path, and its purpose is to allocate the internal resources to the incoming flows, and tracking the state of said flows. This functions are split into three different modules:

- **Flow ID table**:
- **Flow metadata table**:
- **Evict ID queue**:

3.2.2 Packet writer

This stage...

3.2.3 Packet Descriptor and Free Memory queues

Packet descriptor queues This stage Free memory queue This stage

3.2.4 Packet reader

This stage...

Internal vs external memory (HBM usage justification)

One of the first questions regarding system dimensions is the amount of required memory. Based on the aforementioned requirements in 3.1.1, we would like to support a generation size of 64 and at least 128 concurrent active flows. Assuming a worst case size of 1500 bytes per packet, and a packet queue of minimum size (64 since we have to buffer at least as many packets as needed by the generation size); the resulting minimum memory is 12.288 MB assuming perfectly efficient memory usage as seen in the calculation below.

$$\begin{aligned}\text{Memory} &= 64\,\frac{\text{packets}}{\text{flow}} \times 128\,\text{flows} \times 1500\,\frac{\text{bytes}}{\text{packet}} \\ &= 12{,}288{,}000\,\text{bytes} \\ &= \frac{12{,}288{,}000}{10^6}\,\text{MB} \\ &= \boxed{12.288\,\text{MB}}\end{aligned}$$

According to the product page of the Alveo U55C board [3], the total SRAM capacity is roughly 42 MB, of which only around 33 come from the denser UltraRam memory in Xilinx FPGAs. Since the goal of this design is to be potentially scalable to handle hundreds of concurrent flows in the future, building around such a limited pool of storage is not acceptable.

Moreover, the memory is distributed across the FPGA fabric, and assembling it all together would likely require an internal network that would also use a high amount of resources and greatly increase system complexity.

Because of this, the logical decision is to make use of the external memory of the FPGA, which in the U55C's case is the 16 GB worth of HBM.

Coherency and consistency

In computer science and multi-core hardware, coherence ensures that all processors see the latest written value for a single specific memory address, and consistency defines the global order in which read and writes across different memory addresses appear to execute for all processors.

The proposed design does not contain any traditional processor cores, however it does contain two modules (the packet reader and packet writer) that issue and

handle multiple in-flight requests each, raising the question about how to guarantee the coherency and consistency.

Thankfully, one of the biggest advantages of a shared-memory architecture is that the free memory and packet queues guarantee that a packet descriptor can only be at one place at any given time: it is either being processed in the write or read engine, or stored in one of the queues. This ensures that there can only be exactly one request in flight (either a read or a write) for every memory address at any time.

In summary, as long as the write and read engines only pop and push to the queues only when memory transactions have been completed, coherency and consistency are guaranteed.

4 Implementation

This chapter describes the actual deployment of the packet buffer design presented before. The implementation follows a pipelined structure, aiming to map the design into a synthesizable and verified system.

4.1 Context, tools and environment

Before delving into the specifics of the implementation, this section aims to provide the necessary context about the hardware platform and software tools used to develop and evaluate the design.

4.1.1 Systemverilog implementation

Firstly, it is worth mentioning the design has been written entirely in Systemverilog using the synthesizable subset of the language detailed in [9] for the RTL modules. The same language has been used to write the testbench and evaluation environment.

4.1.2 Software tools

The main development, including functional simulations and debugging have been performed using Questa Sim 2023.4. For the out-of-context synthesis flow, Xilinx Vivado 2023.3 was used.

Duck EDA wrapper

As a side note, it is worth mentioning that a Python-based EDA wrapper was developed to simplify and streamline the tasks conducted in this thesis. It allows to easily configure different tests, compilation configurations, and synthesis parameters using YAML configuration files. Its modular design also makes it straightforward to extend with additional tools and functionalities in the future if necessary. More details can be found in the corresponding appendix.

4.1.3 Xilinx Alveo U55C

Previously it was already mentioned that the design is based on the AMD Alveo U55C. This section aims to give a brief overview of its main characteristics. Further

details can be found in the official product Specification [3].

It is an FPGA accelerator card based on the Virtex UltraScale+ XCU55C device, and particularly suited to high-throughput applications because it combines a large pool of FPGA resources with 16 GB of HBM providing 460 GB/s of memory bandwidth and two QSFP28 ports supporting up to 100 Gb/s each. The card connects to the host through PCIe Gen3 x16 or dual Gen4 x8, making it suitable for applications involving both high-speed networking and data movement between the host and FPGA.

Characteristic	**Specification**
FPGA	Xilinx/AMD Virtex UltraScale+ XCU55C
LUTs	1.304 M
Registers	2.607 M
DSP slices	9,024
BRAM	1,968 × 36 Kb (70.9 Mb)
URAM	960 × 288 Kb (270 Mb)
HBM2 capacity	16 GB
HBM2 bandwidth	460 GB/s
Network interfaces	2 × QSFP28, up to 100 Gb/s each
PCIe interface	Gen3 ×16 or 2 × Gen4 ×8
Thermal design power	115 W
Cooling	Passive

Table 4.1: Relevant characteristics of the AMD Alveo U55C accelerator card.

4.1.4 Open-Nic Project Overview

The OpenNIC project [1] is an open-source, FPGA-based network interface card (NIC) platform developed by Xilinx. It provides a programmable hardware architecture for high-performance networking, including a configurable NIC shell, Linux kernel driver, and DPDK support. It is designed to allow developers to implement and evaluate custom networking functions directly on an FPGA, and it is the platfrom in which the design is aimed to be integrated in the future, together with the FEC encoder and the eventual RX packet buffer and FEC decoder.

As shown in As shown in 4.1, the platform offers two slots in the data path to add custom user plugins (i.e. custom modules): the 250 MHz and the 325 MHz box. The main architectural difference between the two is that the 325 MHz box is directly connected to the Gigabit Media Access Control (CMAC) interface, which does not allow back pressure and forces the developer to attend data to be immediately after receiving it in the RX side, whereas the RX and TX adapters buffer incoming and

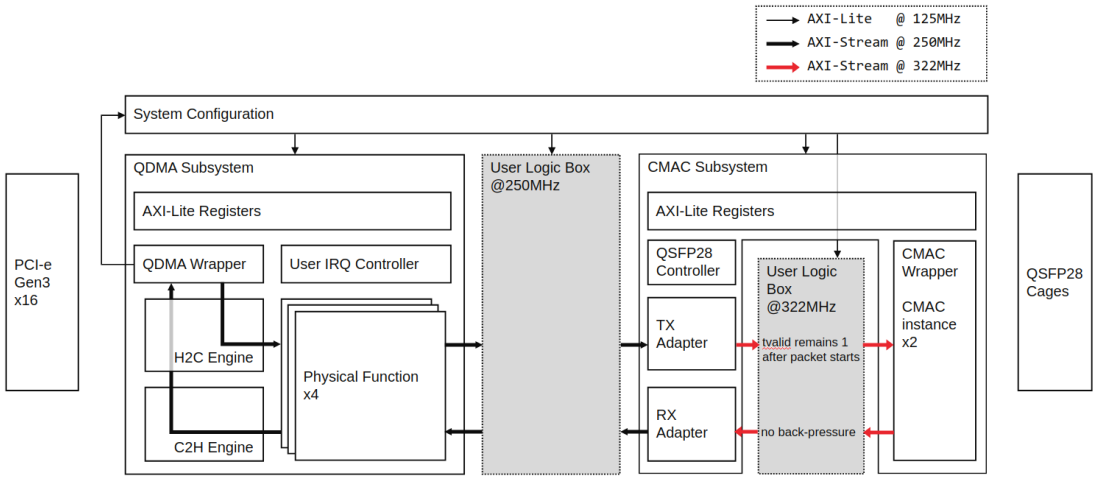

Figure 4.1: OpenNIC Shell architecture (source [1].)

outgoing packets to provide some margin to the logic in the 250 MHz box. Moreover, the lower frequency provides more freedom in the timing side for the design, hence both the FEC encoder and packet buffer are built inside the 250 MHz box.

4.1.5 FEC encoder

Since no final implementation of the FEC encoder is currently available, it is modelled as an abstract entity for the purposes of this design, following the interface defined in 4.2.1. The encoder is assumed to consume packets in groups corresponding to the configured generation size, with all packets within a generation belonging to the same flow. This abstraction allows the remainder of the architecture to be evaluated without depending on the internal implementation or timing characteristics of the encoder.

The FEC encoder is expected to introduce a comparatively large processing latency, potentially in the tens to hundreds of microseconds range. To ensure that this latency dominates the latency budget of the proposed architecture, a target maximum latency of 10 μs is adopted for the remainder of the system. Thus, the memory access and packet-management components are designed such that their contribution remains small relative to the expected FEC processing time.

In practice, the encoder will likely also introduce backpressure when it cannot accept additional packets, and multiple encoder instances could be operated in parallel to increase throughput. However, this parallelism and the resulting encoder back-

pressure are outside the scope of the current implementation; the present design instead assumes an idealized encoder capable of consuming packets at the required generation rate.

4.2 Initial decisions and sizing

Before covering the actual inner workings of the modules, it is important to delve into the characteristics of the interfaces we are working with to see how they will constraint the implementation, particularly in the case of the HBM controller.

4.2.1 Main input and output interfaces (AXI Stream bus)

From an outside perspective, the packet buffer can be thought as a pipeline where the received data is eventually emitted on the other side, only reordered. Hence, it is only natural that the interfaces are the same in the input and output sides.

In the previous section detailing the Open-Nic project architecture 4.1.4, provides the data signals of an AXI-3 Stream bus as the interface with the plugins in the 250 MHz box. The following table 4.2 details said interface, which is common to the input side of the FEC encoder and the packet buffer. The output side of the packet buffer is the mirrored version, where the direction in/out direction inverted.

Signal	Width	Direction	Description
tdata	512	in	Main data signal.
tvalid	1	in	Validates the current tdata value.
tkeep	64	in	Write strobe for the tdata beat. Technically unneeded as the QDMA does not support partial writes.
tlast	1	in	Pulse indicating the last beat of a burst transaction.
tready	1	out	Ready signal from the receiving side.

Table 4.2: AXI-Stream interface signals.

On another note, the clock frequency for the design is 250 MHz as defined in the Open-Nic project. Given the data with of the interface is 512 bits, this gives a maximum theoretical throughput of 128 Gb/s, which is close to the 100 Gb/s maximum the Ethernet MAC can sustain but provides a certain freedom to have some idle cycles without compromising the maximum throughput of the SmartNiC.

$$\text{Throughput} = 512\,\frac{\text{bits}}{\text{cycle}} \times 250 \times 10^6\,\frac{\text{cycles}}{\text{second}} = 128 \times 10^9\,\frac{\text{bits}}{\text{second}} = \boxed{128\,\text{Gbit/s}}$$

Feature	Specification
Memory technology	HBM2
Total capacity	16 GB
Peak HBM bandwidth	460 GB/s
HBM stacks	2
Capacity per stack	8 GB
Pseudo-channels	32 total (16 per stack)
Capacity per pseudo-channel	512 MB
Memory clock frequency	Up to 900 MHz
AXI clock frequency	Up to 450 MHz
AXI interfaces	32 (1 per pseudo-channel)
AXI data width	256 bits (32 bytes)
AXI address width	34 bits
AXI burst length	Up to 16 beats
Minimum transfer size	32 bytes
Addressing modes	Local (non-global) / Global
Access latency (fixed addressing)	90 to 108 memory cycles
Access latency (global addressing)	at least 110 or 128 memory cycles

Table 4.3: Feature overview of the HBM subsystem on the AMD Alveo U55C.

4.2.2 HBM Controller IP overview

The HBM is accessed via the *AXI High Bandwidth Memory Controller LogiCORE IP* [2] provided by Xilinx with their HBM capable boards. Most of the architectural decisions are based on working around its characteristics to meet the throughput and latency requirements, and this section provides a very brief summary of the provided features in table 4.3. More detailed information can be found in the official product page.

4.2.3 HBM Controller operation mode

The HBM controller supports two main operation modes that offer a compromise between ease of use and performance and predictability:

- **Fixed address mode**: Each AXI interface is directly associated with a specific HBM pseudo-channel, and the address range of an AXI port therefore maps directly to its assigned pseudo-channel, avoiding the need for internal crossbar routing. This provides a predictable memory access path with relatively low latency and minimal routing overhead. Particularly, the latency for both read and write requests varies from 90 to 108 memory cycles depending on whether the accessed page was already active. Naturally, the available

memory bandwidth must be managed manually by distributing traffic across the AXI ports and their corresponding pseudo-channels.

- **Crossbar mode**: Each AXI interface can access the full HBM address space, allowing transactions to be dynamically routed to any HBM pseudo-channel through the internal crossbar. This provides greater flexibility in how memory is allocated and accessed, as individual AXI ports are not restricted to a single pseudo-channel. Nonetheless, the additional routing through the crossbar introduces extra latency and contention when multiple AXI interfaces access the same pseudo-channel. A crucial factor in this mode is that according to AMD the latency is unbounded.

Given that the requirements 3.1.1 list a bounded latency of under 10 μs, the fixed address mode is more suitable. Furthermore, the architecture already statically distributes flows across the available memory channels, meaning that the reduced addressing flexibility of fixed address mode does not impose a concerning management workload.

4.2.4 HBM Controller AXI interface decisions and configuration

This section outlines the key high-level architectural and sizing decisions made in the design. These choices establish the overall resource requirements and system structure, providing the foundation for the more detailed implementation decisions discussed in subsequent sections.

Raw throughput considerations and required pseudo-channels

Having seen the interface specifications in [2], the first question that arises is how to address the mismatch of data width: every HBM AXI port (or pseudo-channel) is 256 bits wide, whereas the input and outputs of the packet buffer is 512 bits wide.

On paper every port offers a duplex throughput of 256 bits per cycle, i.e. it can sustain 256 bits per cycle on the write and read sides. Therefore, to achieve an aggregate 512 bit datapath, at least two pseudo-channels operating in parallel are needed.

That being said, it is crucial to consider that the AXI interface can work at up to 450 MHz. This makes it technically possible to maintain the 100 Gbit/s target throughput with only one pseudo-channel as shown in the calculation, specifically, a single 256-bit AXI port operating at 450 MHz provides a theoretical throughput of 115.2Gbit/s, which is sufficient to meet the 100Gbit/s target.

$$
\begin{aligned}
R_{\max} &= 256 \frac{\text{bits}}{\text{cycle}} \times 450 \times 10^6 \frac{\text{cycles}}{\text{s}} \\
&= 115.2 \times 10^9 \frac{\text{bits}}{\text{s}} \\
&= \boxed{115.2\,\text{Gbit/s}} \\
\frac{R_{\text{target}}}{R_{\max}} &= \frac{100\,\text{Gbit/s}}{115.2\,\text{Gbit/s}} = \boxed{86.8\%}
\end{aligned}
$$

However, operating the AXI interface at 450 MHz requires clock-domain crossing (CDC) when interfacing with the 250 MHz processing logic, introducing additional buffering and synchronization complexity. Alternatively, two 256-bit AXI ports operating at 250 MHz provide a combined theoretical throughput of 128Gbit/s without requiring CDC between the processing logic and the HBM interfaces. Although this approach uses an additional AXI port, it provides sufficient throughput margin while keeping the clocking and data path architecture simpler and more deterministic.

Therefore, the two-port, 250 MHz configuration was selected to avoid the additional complexity and potential latency associated with CDC while still comfortably satisfying the 100Gbit/s throughput requirement. Further considerations and potential characteristics of a CDC approach are discussed in 6.

AXI Interface "splitting"

It is frequent to think of AXI ports as a monolithic grouping of the 5 channels described in [4]. This is a fairly natural line of thinking considering that, for example, a traditional processor core will need write and read access to main memory.

However, one of the advantages of the channel design in the AXI protocol is that it is split into independent 3 write (AW, W, B) and 2 read (AR, R) channels, meaning that rather than being thought as a single read/write interface or port, it can be interpreted as two independent ones: a read port and a write port.

Bringing this observation to our design, it is logical to assign the write-related channels to the packet writer and the read-related ones to the packet reader, as these modules will only ever send write and read requests respectively. This is illustrated in figure 4.2.

Latency considerations and HBM memory clock frequency

As explained previously and as detailed in [2], when operating in fixed address mode there is a fixed access latency of 108 or 90 memory cycles to access an inactive and

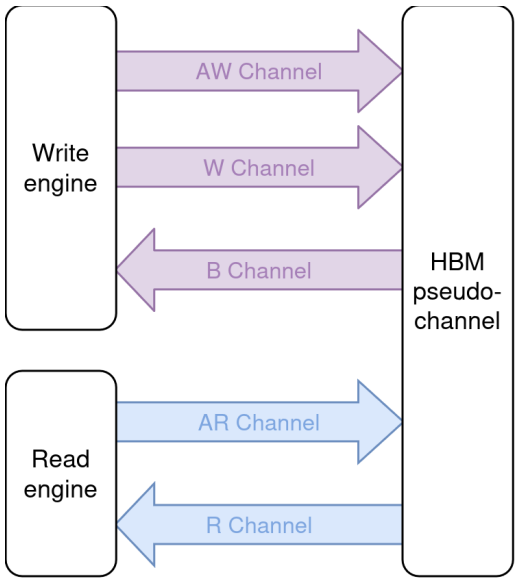

Figure 4.2: AXI Channel splitting into write and read sides (one pseudo-channel example)

active page respectively. It is important to remark that it is measured in memory cycles and not AXI port cycles, therefore the following conversion is necessary to obtain the maximum latency cycles at 250 MHz.

$$T_{\text{HBM}} = \frac{1}{900\,\text{MHz}} = 1.111\,\text{ns},$$
$$T_{\text{open}} = 90 \times T_{\text{HBM}} = 90 \times 1.111\,\text{ns} = 100\,\text{ns},$$
$$T_{\text{closed}} = 108 \times T_{\text{HBM}} = 108 \times 1.111\,\text{ns} = 120\,\text{ns}.$$

$$T_{\text{AXI}} = \frac{1}{250\,\text{MHz}} = 4\,\text{ns},$$
$$N_{\text{open}} = \frac{100\,\text{ns}}{4\,\text{ns}} = \boxed{25\ \text{AXI cycles}},$$
$$N_{\text{closed}} = \frac{120\,\text{ns}}{4\,\text{ns}} = \boxed{30\ \text{AXI cycles}}.$$

This latency can be effectively hidden by maintaining multiple transactions in flight concurrently. Rather than waiting for an individual transaction to complete before issuing the next one, the write and read engines can contiguously issue independent requests while earlier ones are being handled by the HBM controller. This allows the memory pipeline to remain occupied and prevents the access latency from directly limiting the throughput. A quick look at the supported in-flight transactions reveals there is enough margin to cover the latency gap, even without resorting

to bursts:

- 64 in-flight read transactions
- 32 in-flight read transactions

The use of AXI bursts further improves the ability to hide this latency, as a single transaction transfers multiple data beats while occupying only one outstanding transaction slot inside the controller. Moreover, according to the official specification [2] burst lengths greater than one are recommended for improved performance, hence the burst approach was selected.

Available storage space

The discussion so far covered mainly throughput and latency concerns, however the initial reason behind using the HBM available in the U55C was the storage capacity, so this aspect must also be considered.

Anew, considering that 512 MB of storage are allocated to every pseudo-channel, counting 2KB per packet as described in 4.3.1, and assuming two pseudo-channels; yields a total of 500,000 packets, which is enough to fulfil the requirements by one or two orders of magnitude:

$$\text{Total memory} = 2 \times 512\,\text{MB} \quad = 1024\,\text{MB}$$

$$\text{Number of packets} = \frac{1024 \times 10^6\,\text{bytes}}{2 \times 1024\,\text{bytes/packet}} \quad = \boxed{500{,}000\ \text{packets}}$$

Final HBM controller decisions summary

After discussing the requirements and configuration of the HBM controller, the following list aims to sum up the most relevant aspects and decisions taken.

- 250 MHz AXI clock, the same as the processing logic.
- 2 pseudo-channels, one AXI-Stream port for each.
- 256 bits of data width.
- 25 to 30 cycles of latency.
- Bursts of 1 to 16 beats per AXI transaction.
- Write and read transactions completely parallelized because of the port "splitting" in 4.2.

- Pipelined memory access via multiple outstanding AXI transactions.

Other smaller considerations not mentioned in previous discussions are the following:

- No partial write requests allowed, meaning all WSTROBE bits set to 1.
- All memory requests are 32 byte-aligned due to the memory space management described in 4.3.1, meaning the 5 LSB of the requested address set to 0. Added to the fact that packets are aligned in 2KB intervals, it also ensures that only one page is accessed per request due to the maximum of 16 beats per AXI Transaction and the maximum packet size of 1500 bytes.

4.3 Component implementations

From the outside perspective, the packet buffer is conceptually a 512-bit wide pipeline that re-oders the incoming packets into groups of generation size. This section covers the implementation of the different components in the pipeline of the packet buffer. It is important to disclaim that all the diagrams and explanations are simplified representations of the internal architecture, omitting many of the intermediate control and backpressure signals with the aim of showing the main control and data flows across the different stages.

4.3.1 Memory space management

Given that the space in HBM is orders of magnitude over the actually required storage, it is reasonable to allocate 2KB despite packets being at most 1500 bytes long. Although this leaves a notable amount of memory unused, it also achieves simpler memory addressing logic by keeping the lower bits of the address always to 0. In the future, other more efficient schemes (such as fragmentation) can be pursued if total storage becomes a concern.

The two HBM pseudo-channels are selected using the LSB of the packet address provided by the free memory queue (in the packet writer's case) or the packet queues (in the packet reader's case). Since memory locations are distributed across the two pseudo-channels, the LSB can be used as a simple deterministic mapping: one value selects the first pseudo-channel, while the other selects the second.

This avoids the need for additional channel-selection logic and naturally distributes allocations across the two pseudo-channels, provided that the free-memory addresses are appropriately interleaved. This interleaving is not guaranteed and can

lead to slightly unpredictable performance as time goes on. Nonetheless, the natural randomness in packet arrival when it comes to the length of bursts of same-flow packets is expected to keep performance relatively constant.

Although potentially obvious, another advantage of this distribution is that packets are contiguously stored in memory and always in the same 4KB page, which provides good access patterns to the HBM memory when added to the usage of burst transactions.

4.3.2 Flow ingest

This flow ingest 4.3 is the first stage in the pipeline; here actual flow tuples are assigned an internal identifier, which determines which queue will store this flow's packet addresses.

Since the packet buffer can handle a limited number of concurrent flows, it is necessary to track the state of each one to reallocate the resources to new flows once older ones become inactive. This task is handled by the scoreboard, which contains a small FSM per flow to determine their state. Flows are considered inactive when all its packets have been fed to the encoder are none more are queued anywhere in the pipeline, or after a set amount of time without receiving packets.

When a flow is declared inactive by the scoreboard, it will eventually push its allocated identifier to the free ID queue, allowing the tuple table to assign it to a different tuple.

Tuple table

The tuple table is a simple structure that assigns flow tuples to internal IDs. It pops inactive ones from the queue and reports a hit to the scoreboard upon a packet arrival when either its tuple already had an ID assigned, or it is given a new one.

For simplicity and predictability, a straightforward dictionary-based structure is used for the tuple table rather than a more scalable hash-based implementation. Although a hash table would provide better scalability for numerous entries, the expected number of active flows in the proposed architecture is right at the edge of being small enough that the additional complexity of hashing, collision handling, and table management is not justified. The dictionary structure therefore provides a simpler implementation and control logic while remaining adequate for the target workload.

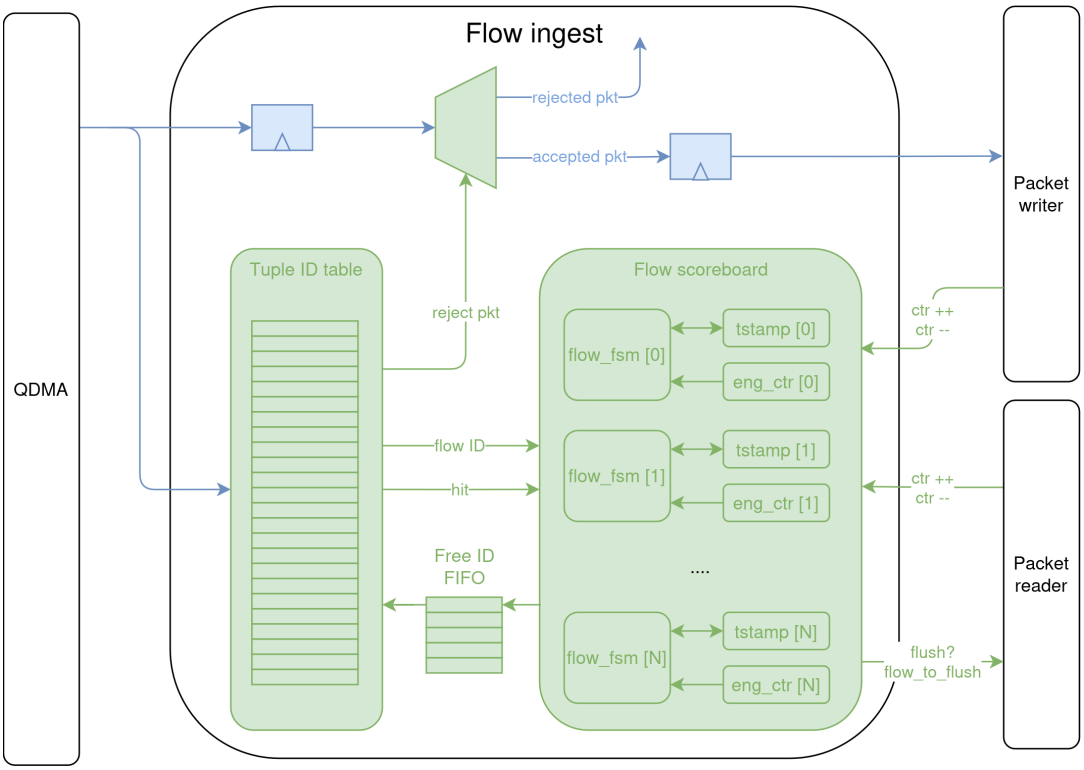

Figure 4.3: Flow ingest stage overview

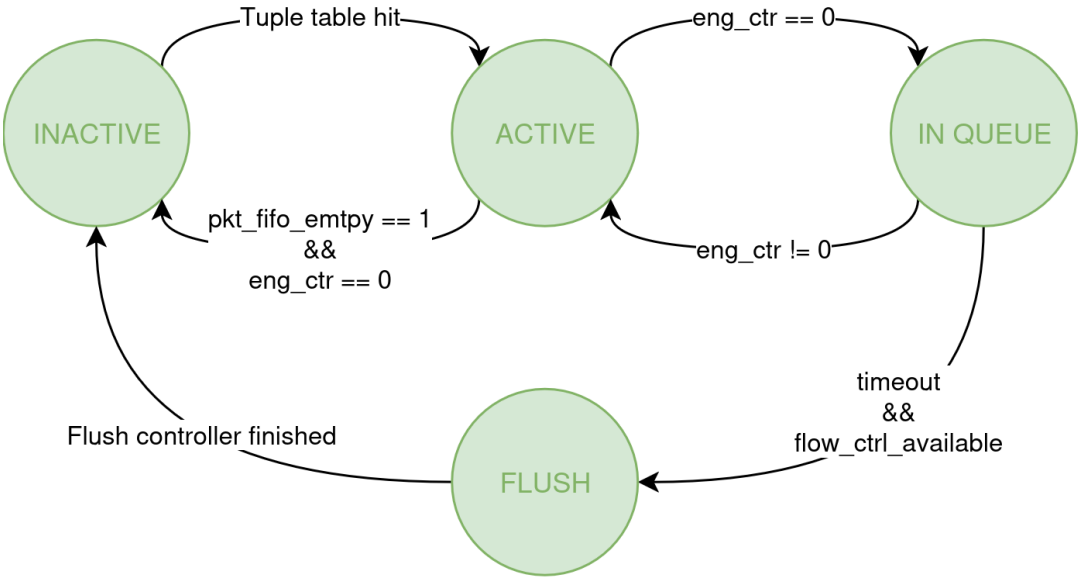

Figure 4.4: Flow state diagram

Scoreboard

Due to the limited number of active flows there is a necessity to internally track the state of all the concurrent flows. Every flow has a slot in the scoreboard where its state is tracked following the state machine in 4.4.

A flow is declared active when its first packet arrives and a new ID is allocated to it in the tuple table, and every packet arrival within a flow resets its timestamp counter. It can remain in this state indefinitely as long as new packets keep arriving from the QDMA or are being fed to the encoder. When all the packets are in the queues it transitions to the waiting state, where if not selected by the arbiter (due to backpressure or insufficient number of packets) will eventually be declared inactive and its ID will be returned.

Due to the pipelined design, there are some sections where a packet is being written or read from memory. To avoid memory hazards, a flow is protected (transitioning it to the active state) when the write or read engine are processing packets associated to it, meaning it can not be declared inactive. This is what the engine counter tracks, i.e. the number of packets in either of the engines, therefore if the counter is not 0 the flow will never transition to the flush state. The counter increases at the exact time a packet enters either the packet reader or writer, and decreases once it exits. Summarizing:

- **Inactive**: That particular flow ID is not being used. It can be pushed to the free ID queue.

- **Active**: The ID is allocated, and an associated packet is currently being processed at the packet reader or writer.

- **Wait**: The flow has packets queued, but none are in either the packet writer or reader.

- **Flush**: Due to timeout, the packets from that FIFO are being flushed and returned to the free mem queue.

4.3.3 Packet writer

The packet writer is the stage where the incoming data is written into memory. Here a completely separate control and data paths are created by means of the burst instruction detailed below, which is the intermediate step between packet and actual AXI transactions.

Overall, as seen in 4.5 this stage is divided into the instruction encoder, which builds the burst instructions for the engines; the write engines themselves, which issue AXI write transactions to the HBM; and the re-oder FIFO, which keeps track of the issue order of packets between the engines. As the engines finish writing, the start address and size of the packet (information provided by the engines) is pushed to the corresponding packet queue (determined by the re-order FIFO).

Regarding the burst instructions, when a new packet is provided by the previous stage the encoder calculates how many AXI transactions (and how many beats per transaction) are needed to write the full packet to memory. This is done by inspecting the packet bytes in the IP header, since it is always the same size and at the same offset. Apart from that, based on the head of the free memory FIFO the address of every transaction is computed, and the pseudo-channel channel to write to is selected by the LSB of the free memory address. The final format of the instruction can be seen below.

As a side note, it is worth mentioning that the majority of the size calculations are fairly inexpenisve in terms of hardware resources, as it is mostly divisions by powers of 2, which can be built simply via bit-shifting the data.

`start_addr`	`num_beats`	`is_last_burst`
34 bits	5 bits	1 bit

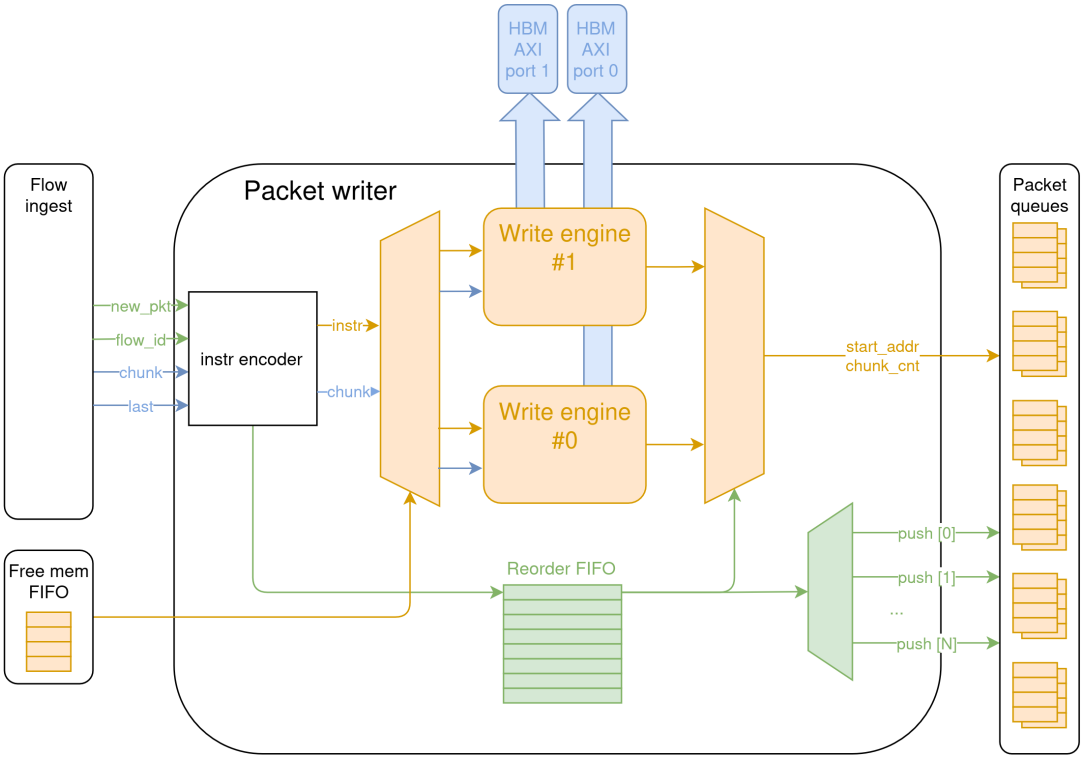

Figure 4.5: Packet Writer overview

- `start_addr`: Byte address of the burst. The first burst of each packet is 2 KB aligned, while subsequent bursts are 64 B aligned.
- `num_beats`: Number of 256-bit beats contained in the burst, with a range of 1–16 beats.
- `is_last_burst`: Indicates whether the burst is the final burst belonging to the packet.

On the data side, the LSB of the head of the memory FIFO selects which write engine to push the data to. When a new packet is detected, the selected pseudo-channel (and write engine) changes accordingly. It is important to issue the burst instruction right when the first beat of the packet arrives from the previous stages, this way the AXI transaction can be streamed immediately rather than having to wait for the full packet to arrive. This approach does not only streamline the latency by avoiding stall cycles while the packet arrives, but it also greatly reduces resource usage as there is no need to buffer an entire package before starting to write it to the HBM.

Having two write engines operating at the same time opens the door to potential order violations, hence a mechanism to track packet issue order is necessary. As an example, let us say packet A and B belong to the same flow and arrive in that order. A is issued to pseudo-channel 0 and B to pseudo-channel 1, and also B is notably smaller than A. If there is backpressure on pseudo-channel 0 (due to previous in-flight writes for example) it is highly probable that B will be fully written before A, so B would be pushed to the packet queue before A, therefore violating packet arrival order.

In order to prevent this, the re-oder FIFO tracks which is the next channel to expect a packt from, and which packet queue it is meant to be pushed to. It is worth mentioning that currently it is assumed that packet writes will be completed in the same order as they were issued (the write engines use the same AXI ID in all transactions to enforce it).

In summary, when a new packet arrives the packet writer assigns it a pseudo-channel, encodes the AXI burst instructions for the corresponding write engine, and pushes an entry to the re-oder FIFO to track the new outstanding write. Once the engine is finished, the packet is pushed to the flow packet FIFO.

Write engine

Figure 4.6 contains an architectural overview of the packet writer: when the burst instruction is issued by the packet writer, the push to the FIFO triggers the AW

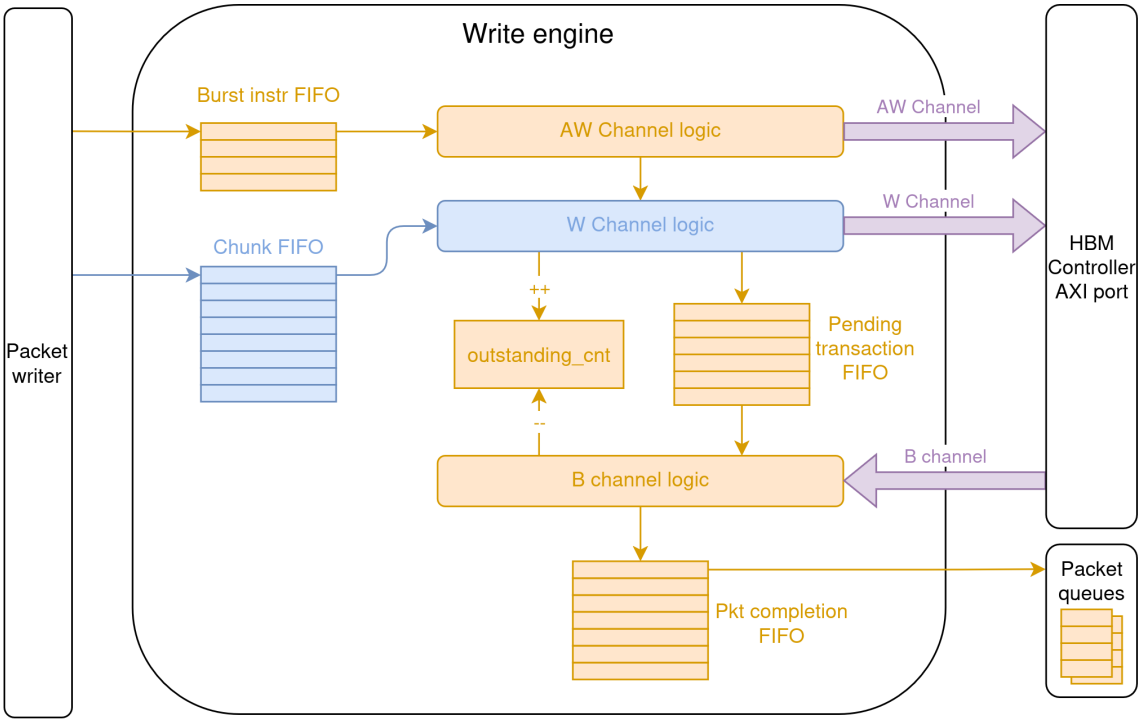

Figure 4.6: Write engine overview

channel logic to issue a new transaction. Once the handshake occurs in that channel, the W channel logic activates, sending as many beats of data as indicated in the burst instruction. It is important to remark that the chunk FIFO is pushed two entries at a time (each entry is 256 bits wide), but every AXI beat just pops one entry. This means the read speed is only half of the write speed, hence why there are two engines present.

Once all the beats are sent, a new entry is pushed to the pending packet FIFO. This second queue tracks transactions which have been correctly issued but are not yet completed, and decouples the request and response sides. This means that the current burst instruction can be popped as it has been already processed without having to wait for the AXI response, therefore creating an AXI transaction pipeline.

Once the AXI response is received, the pending transaction is popped from the queue. Then the B channel logic is tasked with re-building the packet metadata (base address and total 512-bit chunks) that will be pushed to the packet queues by re-counting what transactions were part of the same packet and how many total beats were issued. Once this information is re-built, it is pushed into the completion FIFO, which will be popped by the packet writer to finally push the entry into the

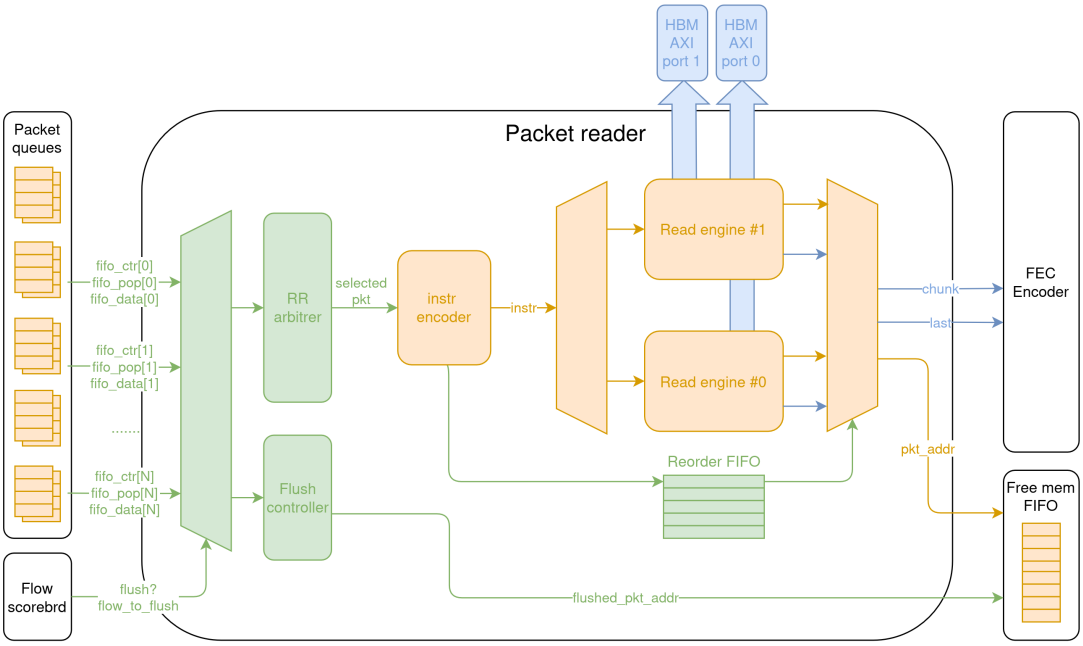

Figure 4.7: Packet reader overview

corresponding queue.

4.3.4 Packet reader

Needless to say, the packet reader shown in 4.7 is the counterpart to the write engine, handling the read side of the HBM memory.

Firstly, the module filters out the flow to be flushed by the flush controller as instructed by the scoreboard, which is keeping track of the state of all the flows in the system. Flushing the flow returns the packet addresses to the free memory queue without reading them from memory, therefore dropping the stored data and freeing the location to be overwritten later by a new packet.

The rest of flows are fed to the round-robin arbiter, which computes what flows are eligible to be fed to the encoder by counting the number of packets present in the queues. Any flow with a queue containing the generation size number of packets or greater is considered valid.

The arbiter then selects a valid flow and pops a generation size worth of packets to feed them to the instruction encoder. The instruction encoder computes the same burst instructions as in the write engine and feeds them to the corresponding read

engine, again following the same criteria of using the LSB of the packet address to select the channel. At the same time as the instructions are fed, a new entry is added to the re-oder FIFO.

Both read engines contain a queue storing the read data (and indicating packet boundaries with a "last" bit), which are popped to be fed to the FEC encoder following the order determined by the re-order FIFO. Once a packet is fully streamed to the FEC encoder, the address is returned to the free memory queue. Given that memory transactions are completed in-order, the packets are also guaranteed to be served in order and in groups of generation size belonging to the same flow. Also, the re-oder FIFO tracks the flow ID of the issued packets to decrease the engine counter in the scoreboard later on.

Once the arbiter counts up to a generation size worth of packets, the arbitration is restarted, shifting the priority to attend a different flow. It is important to remark that packets are streamed from the queues to the read engine to maintain a pipeline structure, instead of being read and buffered intermediately.

Flush controller

Currently, only flows with a number equal or greater than the encoder's generation size can be issued by the arbiter to be read from the HBM. Ideally, remaining packets would be padded with dummy data and sent to the encoder, but this mechanism has not been implemented yet. In its place, a placeholder flush controller has been implemented, whose only task is to feed stale packet addresses back to the free memory queue.

The flush controller is a simple FSM that receives instructions directly from the flow scoreboard. It receives a flow ID, which indicates what queue should be flushed and a valid pulse which signals the start of the process. Once given the signal to start, it pops one packet entry per cycle and pushes the start address to the free memory queue. The free memory queue has only a single write port, so either the flush controller or the engines can write to it at once. For this design, the selected approach has been to prioritize the main data flow, relegating the flush controller to push to the queue only when the engines are not.

Round-robin arbiter

The arbiter in the system follows a round-robin arbitration policy, ensuring that flows are served in a fair and deterministic manner by rotating the priority among the requesting agents. The specific implementation details and algorithm used to

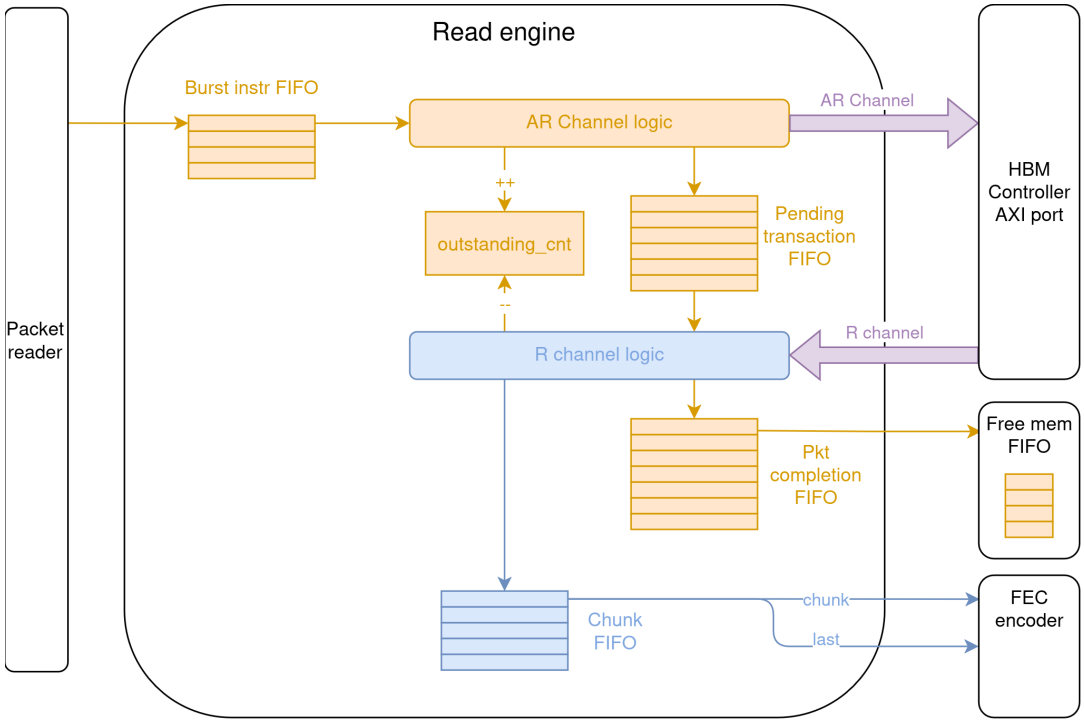

Figure 4.8: Read engine overview

realize the round-robin policy are beyond the scope of this work. A detailed description of the underlying algorithm can be found in LaForest's A Round-Robin Arbiter [5].

It is worth noting that the implementation of the arbiter allows assigning different priority weights to all the inputs to be arbitrated, meaning that different tiers of priority can be created inside the system to prioritize critical data to be sent first; however, these inputs are not used currently, so all flows have an equal priority as of now.

Read engine

The read engine 4.8 follows a very similar structure to the write engine, pipelining the AXI requests across the channels.

When a burst instruction is pushed to the queue, the AR channel logic checks if there is enough space in the next queues to house the data that will be read from memory. When there is enough space, it issues the read request to the HBM. Once

the handshake is completed, it pushes the in-flight request to the pending queue and issue a new read request to keep the transaction pipeline going.

The R channel logic then takes the pending requests and the read data from the HBM and feeds it to the chunk FIFO, which will be later popped by the FEC encoder. In parallel, the start address is pushed to the completion queue once the last beat of the last transaction of a given packet is received. This address is later returned to the free memory queue.

In simmetry to the write engine, the chunk FIFO in this case is pushed one 256-bit entry at a time (the width of the HBM AXI interface data lanes), and it is popped 512 bits at a time (the width of the FEC encoder stream interface).

4.3.5 Packet and free memory queues

As stated in the design chapter 3, there is one packet queue per flow and a global free memory queue. Despite not being the most scalable solution, for this first version of the design simple single-port flip-flop based FIFOs have been used.

Ideally the free memory queue should contain one entry for every packet location available in the system.

$$N_{\text{entries}} = N_{\text{flows}} \times N_{\text{pkt/flow}}.$$

For example, with 128 flows and 64 packet buffers allocated per flow, the queue would require:

$$N_{\text{entries}} = 128 \times 64 \quad = \boxed{8192 \text{ entries}}.$$

Maintaining such a large FIFO is impractical, as it would require thousands of entries and introduce significant storage and control overhead. Because of this, the free memory queue is another configuration parameter whose effect will be discussed later on.

5 Evaluation

Using the previously presented implementation, you can now evaluate your approach.
Get as many quantitative measurements as possible. E.g. compare different implementations in terms of hardware usage, latency...
This chapter is usually filled with graphs and numbers. In the end, the numbers should show how your approach is better (in some regard) than existing approaches.

5.1 Performance evaluation

5.1.1 Testbench architecture

Briefly mention the almost UVM-like structure of the testbench, and the stuff it allows us to do.

5.1.2 Test suite and configs

Comment the axis of freedom (NR_FLOWS, GENSIZE, PKT_FIFO_LENGTH, MEM_FIFO_LENGTH... and showcase results.

There will be a table for every cobination of the previous parameters we deem appropriate.

Here are some flow patterns we test with every configuration:

- Random burst length, small pkts.
- Random burst length, Big pkts.
- Random burst length, Random pkts.
- Long burst length, R andom pkts.
- ...

6 Discussion

Hello.

7 Conclusion and Outlook

Conclude Summarize the work you did. Show how well you reached the goal you specified in the introduction.

Outlook Which new questions arise based on your results? If you would continue to work on this topic, what would your next work item be?

8 Appendix

Hello

Bibliography

[1] AMD. AMD OpenNIC Project. https://github.com/Xilinx/open-nic. Accessed: 2026-08-14.

[2] AMD. *AXI High Bandwidth Memory Controller LogiCORE IP Product Guide (PG276)*. AMD, 2025. Accessed: 2026-08-12.

[3] AMD. AMD Alveo™ U55C High Performance Compute Card. https://www.amd.com/en/products/accelerators/alveo/u55c/a-u55c-p00g-pq-g.html, 2026. Accessed: 2026-08-12.

[4] Arm Limited. AMBA AXI and ACE Protocol Specification: Channel Signals, 2026.

[5] Charles LaForest. A round-robin arbiter. https://fpgacpu.ca/fpga/Arbiter_Round_Robin.html, 2026. FPGA Design Elements.

[6] M. Lau et al. Gigabit ethernet switches using a shared buffer architecture. *IEEE Communications Magazine*, 41(12):76–84, December 2003.

[7] Jad Naous, Glen Gibb, Sara Bolouki, and Nick McKeown. Netfpga: Reusable router architecture for experimental research. In *Proceedings of the 2008 ACM SIGCOMM Workshop on Programmable Routers for Extensible Services of Tomorrow (PRESTO)*, pages 1–6. ACM, August 2008.

[8] S. O'Kane, C. McKillen, and Sakir Sezer. The design and implementation of a shared packet buffer architecture for fixed and variable sized packets. 2005:352 – 356, 08 2005.

[9] Stuart Sutherland. A proposal for a standard synthesizable subset for systemverilog-2005: What the ieee failed to define. In *Design Verification Conference (DVCon)*, San Jose, California, February 2006.

[10] Mark Watson, Ali Begen, and Vincent Roca. Forward error correction (fec) framework. RFC 6363, Internet Engineering Task Force, October 2011.

Confirmation

Herewith I, Andreu Roca i Montserrat, confirm that I independently prepared this work. No further references or auxiliary means except those declared in this document have been used. I have used generative AI tools for the following purposes:

- literature research
- coding
- automation

Munich, August 24, 2026

.....
Andreu Roca i Montserrat